

Pflichtenheft – Service: company-service

Service:	company-service
Version:	1.4
Stand:	25.02.26
Technologie:	Java 21, Spring Boot 4.0.x, Maven, MariaDB/MySQL, Flyway, REST/JSON, OpenAPI/Swagger (profilabhängig), Docker
Root-Package:	de.innologic.companyservice
Datenbank:	company (Schema/DB-Name konfigurierbar)
API Context Path:	/api/v1
Effektive Basis-URL (Default):	http://<host>:8110/api/v1
API/Application:	SERVER_PORT (Default 8110)
Management/Actuator:	COMPANY_MANAGEMENT_SERVER_PORT (Default 8110)

Inhaltsverzeichnis

1. Ziel und Zweck.....	4
2. Abgrenzung.....	4
2.1 Enthalten.....	4
2.2 Nicht enthalten.....	4
3. Systemkontext / Abhängigkeiten.....	5
3.1 Inbound (wer ruft company-service auf).....	5
3.2 Outbound (wen nutzt company-service).....	5
3.3 Verantwortlichkeiten (Source of Truth).....	5
3.4 Datenflüsse (Onboarding / Docking / Löschung).....	5
3.4.1 Onboarding (Bootstrap).....	5
3.4.2 Docking (Kontakt/Kommunikation).....	5
3.4.3 Löschung (Deletion Workflow).....	6
4. Begriffe.....	6
5. IAM-Absicherung und Mandantentrennung.....	6
5.1 Grundprinzip (OAuth2 Resource Server).....	6
5.2 JWT-Contract (Claims, Audience, Subject Types).....	6
5.3 Tenant-Kontext und Konsistenzprüfungen.....	7
5.4 Autorisierung über Scopes (Scope-Matrix, vollständig).....	7
6. REST-API – Übersicht.....	8
6.1 Basis-URL, Versionierung, Content Types.....	8
6.2 Endpunkte (fachlich).....	8
6.3 Standard-Query (Paging/Sort/Filter).....	9
7. Funktionale Anforderungen (Use Cases).....	9
7.1 Company anlegen (Bootstrap).....	9
7.2 Company lesen/ändern.....	10
7.3 Headquarter verwalten.....	10
7.4 Locations anlegen/listen/ändern.....	10
7.5 Location schließen / wieder öffnen.....	10
7.6 Company löschen (Deletion Workflow).....	10
7.7 Deletion-Status abfragen.....	10
7.8 Jurisdiktion (timezone/countryCode/regionCode).....	11
7.9 Docking-Contract (OwnerKey).....	11
8. Datenmodell (Persistenz).....	11
8.1 Company.....	11
8.2 Location.....	11
8.3 DeletionTombstone.....	11
8.4 OutboxEvent (optional).....	11
9. Idempotency-Regeln.....	12
10. Fehlerformat, Statuscodes, Fehlercodes.....	12
10.1 Fehlerformat (API Error).....	12
10.2 Statuscodes.....	12
10.3 Fehlercode-Mapping (Fehlernummer, Fehlermeldung, Auslöser).....	12
11. Betrieb / Deployment / Konfiguration.....	14
11.1 Serviceeigene Ports (nur Defaults, ausschließlich per ENV konfigurierbar).....	14
11.1.1 API/Application Port.....	14
11.1.2 Management/Actuator Port.....	14
11.1.3 Optionale Ports (nur Dev/Test; Default deaktiviert).....	14
11.1.4 Regel bei parallelem Betrieb mehrerer Services auf einem Host.....	14
11.2 Pfad-Konfiguration (nur per ENV).....	14
11.3 JWT Resource Server – Konfiguration (nur per ENV).....	15

11.4 Umgebungsvariablen.....	15
12. Tests und Abnahmekriterien.....	16
12.1 Tests.....	16
12.2 Abnahmekriterien.....	17

1. Ziel und Zweck

Der company-service verwaltet Company-Stammdaten (Mandant) und Locations (Standorte) innerhalb einer mandantenfähigen Plattform.

Schwerpunkte:

- Company-Metadaten und Locations verwalten
- Hauptstandort (Headquarter) pro Company über `company.mainLocationId`
- Location-Status `OPEN/CLOSED` mit Regeln für `close/reopen`
- Jurisdiktion auf Location-Ebene: `timezone`, `countryCode`, `regionCode`
- Mandantentrennung über `tenant_id` im JWT
- Docking-Contract für Kontakt- und Kommunikationsdaten über OwnerKeys
- Löschung über orchestrierten Deletion Workflow (Hard-Delete)

2. Abgrenzung

2.1 Enthalten

- CRUD für Company und Location (inkl. Listen mit Paging/Sort/Filter)
- Headquarter-Funktion (lesen/setzen)
- Statuswechsel `close/reopen`
- Jurisdiktion-Felder inkl. Normalisierung/Validierung
- JWT-Validierung inkl. Audience-Check, Scopes und Mandantentrennung
- Deletion Workflow inkl. technischem Tombstone und Events (optional über Outbox)

2.2 Nicht enthalten

- Token-Ausstellung (Login/MFA/Refresh) – auth-service
- Benutzerverwaltung/Registrierung – user-service / auth-service
- Kontaktpunkte/Adressen (E-Mail, Telefon, postalische Adresse) – contact-service
- Versand/Retry/Delivery-Handling – communication-service
- Feiertags-/Arbeitsregelberechnung – separater Rules/Holiday Service

3. Systemkontext / Abhängigkeiten

3.1 Inbound (wer ruft company-service auf)

- Portal/Backend (User Token): Lesen/Ändern im eigenen Tenant
- Interne Services (Service Token): Provisionierung, Automationen, Administration
- auth-service (Service Token): Bootstrap-Anlage im Onboarding/Registrierungsflow

3.2 Outbound (wen nutzt company-service)

- auth-service: JWKS/Issuer als Quelle zur JWT-Validierung (Resource-Server Betrieb)
- Event-Bus/Message-Broker (optional): Publikation von Domain-Events (über Outbox)

3.3 Verantwortlichkeiten (Source of Truth)

- company-service: Company-Metadaten + Locations + Jurisdiktion + Headquarter (`mainLocationId`)
- contact-service: Adressen, ContactPoints, Owner-Modell (`ownerType`, `ownerId`)
- communication-service: Versandaufträge/Delivery/Retry; Empfängerauflösung über contact-service
- iam-service: Rollen-/Rechteverwaltung und Zuweisungen (wirkt über Scopes/Rollen im Token)
- user-service: Benutzerprofil und user-bezogene Metadaten (keine Userdaten im company-service)

3.4 Datenflüsse (Onboarding / Docking / Löschung)

3.4.1 Onboarding (Bootstrap)

1. auth-service stellt ein Service-Token aus, das `aud=["company-service"]` und passende Scopes enthält.
2. auth-service ruft `POST /api/v1/companies` auf und übergibt Company-Daten inkl. initialer Location.
3. company-service speichert Company + erste Location atomar und setzt `mainLocationId`.

3.4.2 Docking (Kontakt/Kommunikation)

1. company-service liefert OwnerKeys in Responses zurück (`contactOwnerType`, `contactOwnerId`).
2. contact-service speichert ContactPoints/Adressen zu OwnerKeys.
3. communication-service löst Empfänger über OwnerKeys beim contact-service auf.

3.4.3 Löschung (Deletion Workflow)

1. Ein administrativer Aufruf startet `DELETE /api/v1/companies/{companyId}`.
2. `company-service` erzeugt einen `DeletionTombstone` und publiziert ein `Deletion-Event` (Outbox, optional).
3. Abhängige Services löschen Folgedaten und bestätigen (Ack).
4. `company-service` finalisiert den Workflow und entfernt Company-Daten.

4. Begriffe

- Company: Mandant/Unternehmen (Stammdaten)
- Location: Standort einer Company
- Headquarter / Hauptstandort: eine Location pro Company, definiert über `company.mainLocationId`
- Status (Location): `OPEN` oder `CLOSED`
- Jurisdiktion: `timezone + countryCode + regionCode`
- `tenant_id`: Mandantenkennung im JWT; Grundlage der Mandantentrennung
- OwnerKey: `ownerType + ownerId`
 - Company: `COMPANY + companyId`
 - Location: `LOCATION + locationId`
- Idempotency-Key: Header zur Wiederholbarkeit ausgewählter Schreiboperationen

5. IAM-Absicherung und Mandantentrennung

5.1 Grundprinzip (OAuth2 Resource Server)

Alle fachlichen Endpunkte erwarten `Authorization: Bearer <JWT>`.

Absicherung folgt dem Default-Deny-Prinzip: Routen ohne explizite Regel sind nicht erreichbar.

5.2 JWT-Contract (Claims, Audience, Subject Types)

JWT-Prüfungen:

- Signaturprüfung über `issuer-uri` oder `jwk-set-uri`
- Zeitvalidierung (`iat`, `exp`) mit Clock-Skew
- Audience-Check: `aud` enthält `company-service`
- Subject-Type: `subject_type` ist `USER` oder `SERVICE`
- Tenant-Claim: `tenant_id` ist vorhanden (für tenant-scharfe Endpunkte)
- Scopes: `scp` (Liste) oder `scope` (String)

Deterministische Contract-Fehlermeldungen (401), z. B.:

- `missing claim: tenant_id`
- `missing claim: scp`
- `wrong audience`
- `invalid subject_type`

5.3 Tenant-Kontext und Konsistenzprüfungen

- Tenant-Quelle ist ausschließlich `tenant_id` aus dem JWT.
- Optionaler Konsistenzheader: `X-Tenant-Id`
 - bei Abweichung gegenüber `tenant_id` erfolgt Ablehnung (403).
- Pfadrouten mit `{companyId}`: Vergleich `companyId == tenant_id`
- ID-basierte Location-Routen: Tenant wird serverseitig über `location.companyId` aus DB geprüft und mit `tenant_id` verglichen.
- Optionaler No-Data-Leak Modus:
 - bei Zugriff auf fremde Location-by-id wird 404 geliefert (statt 403).

5.4 Autorisierung über Scopes (Scope-Matrix, vollständig)

Alle fachlichen Funktionen des company-service sind über Scopes abgesichert:

Bereich	HTTP	Route	Scope
Bootstrap	POST	<code>/api/v1/companies</code>	<code>company:create</code>
Company	GET	<code>/api/v1/companies/{companyId}</code>	<code>company:read</code>
Company	GET	<code>/api/v1/companies</code>	<code>company:read</code>
Company	PUT/PATCH	<code>/api/v1/companies/{companyId}</code>	<code>company:write</code>
Company Logo	PUT	<code>/api/v1/companies/{companyId}/logo</code>	<code>company:write</code>
Company Logo	DELETE	<code>/api/v1/companies/{companyId}/logo</code>	<code>company:write</code>
Headquarter	GET	<code>/api/v1/companies/{companyId}/headquarter</code>	<code>company:read</code>
Headquarter	PUT	<code>/api/v1/companies/{companyId}/headquarter</code>	<code>company:admin</code>
Hauptlocation (Legacy)	PUT	<code>/api/v1/companies/{companyId}/main-location</code>	<code>company:admin</code>
Company löschen	DELETE	<code>/api/v1/companies/{companyId}</code>	<code>company:admin</code>
Deletion Status	GET	<code>/api/v1/companies/{companyId}/deletion-status</code>	<code>company:admin</code>
Locations	GET	<code>/api/v1/companies/{companyId}/locations</code>	<code>company:read</code>
Locations	POST	<code>/api/v1/companies/{companyId}/locations</code>	<code>company:write</code>
Location	GET	<code>/api/v1/location/{locationId}</code>	<code>company:read</code>
Location	PUT/PATCH	<code>/api/v1/location/{locationId}</code>	<code>company:write</code>
Location	POST	<code>/api/v1/location/{locationId}/close</code>	<code>company:admin</code>
Location	POST	<code>/api/v1/location/{locationId}/reopen</code>	<code>company:write</code>
Location	DELETE	<code>/api/v1/location/{locationId}</code>	<code>company:admin</code>

Bootstrap ist zusätzlich eingeschränkt:

- `subject_type=SERVICE`
- `sub=auth-service` (Service-Principal)

6. REST-API – Übersicht

6.1 Basis-URL, Versionierung, Content Types

- API Context Path: `/api/v1`
- Request/Response: `application/json`
- Fehler: `application/json` (siehe Abschnitt 10)

Standard-Header:

- `X-Correlation-Id` (optional; wird sonst erzeugt und zurückgegeben)
- `Idempotency-Key` (für definierte Endpunkte, siehe Abschnitt 9)
- `X-Tenant-Id` (optional; Konsistenzcheck, siehe Abschnitt 5.3)

6.2 Endpunkte (fachlich)

Company:

- `POST /api/v1/companies`
- `GET /api/v1/companies/{companyId}`
- `GET /api/v1/companies`
- `PUT /api/v1/companies/{companyId} / PATCH /api/v1/companies/{companyId}`
- `PUT /api/v1/companies/{companyId}/logo`
- `DELETE /api/v1/companies/{companyId}/logo`
- `DELETE /api/v1/companies/{companyId}`
- `GET /api/v1/companies/{companyId}/deletion-status`

Headquarter:

- `GET /api/v1/companies/{companyId}/headquarter`
- `PUT /api/v1/companies/{companyId}/headquarter`

Legacy:

- `PUT /api/v1/companies/{companyId}/main-location`

Locations:

- GET /api/v1/companies/{companyId}/locations
- POST /api/v1/companies/{companyId}/locations
- GET /api/v1/location/{locationId}
- PUT /api/v1/location/{locationId} / PATCH /api/v1/location/{locationId}
- POST /api/v1/location/{locationId}/close
- POST /api/v1/location/{locationId}/reopen
- DELETE /api/v1/location/{locationId}

6.3 Standard-Query (Paging/Sort/Filter)

Paging/Sort:

- page (default 0)
- size (default 50; max z. B. 200)
- sort (z. B. createdAt, DESC)

Filter (Locations):

- status=OPEN|CLOSED
- nameContains=<text>

7. Funktionale Anforderungen (Use Cases)

7.1 Company anlegen (Bootstrap)

- Persistiert Company + initiale Location atomar
- Setzt company.mainLocationId auf initiale Location
- initiale Location startet mit status=OPEN

Beispiel Request:

```
{
  "name": "InnoLogic GmbH",
  "displayName": "InnoLogic",
  "timezone": "Europe/Berlin",
  "locale": "de-DE",
  "logoFileRef": "file_abc123",
  "initialLocation": {
    "name": "Bremen HQ",
    "locationCode": "HB-01",
    "timezone": "Europe/Berlin",
    "countryCode": "DE",
    "regionCode": "DE-HB"
  }
}
```

7.2 Company lesen/ändern

- Lesen liefert Company inkl. `mainLocationId`
- Änderungen umfassen Metadaten (z. B. `name`, `displayName`, `timezone`, `locale`, `logoFileRef`)
- Parallelitätsschutz über `version` (Optimistic Lock)

7.3 Headquarter verwalten

- Abfrage liefert `locationId` des aktuellen Hauptstandorts
- Setzen erfolgt auf eine Location derselben Company
- Ziel-Location hat `status=OPEN`

Beispiel Response:

```
{ "locationId": "01J3Z51A7B2C3D4E5F6G7H8J9K" }
```

7.4 Locations anlegen/listen/ändern

- Anlegen setzt Default `OPEN`
- Listen unterstützt Paging/Sort/Filter (Abschnitt 6.3)
- Änderungen umfassen z. B. `name`, `locationCode`, `timezone`, `countryCode`, `regionCode` + `version`

7.5 Location schließen / wieder öffnen

Schließen:

- Headquarter bleibt geöffnet
- Nach dem Schließen verbleibt mindestens eine Location im Status `OPEN`

Beispiel Request (close):

```
{ "reason": "Zusammenlegung / Umzug" }
```

Wieder öffnen:

- Setzt `CLOSED` zurück auf `OPEN`

7.6 Company löschen (Deletion Workflow)

- `DELETE /companies/{companyId}` startet asynchronen Löschprozess und liefert `202 Accepted`
- Während `IN_PROGRESS` gilt die Company fachlich als nicht vorhanden (GET liefert `404`)

7.7 Deletion-Status abfragen

- `GET /companies/{companyId}/deletion-status` liefert `state` sowie Zeitpunkte/Details (inkl. Fehlertext bei `FAILED`)

7.8 Jurisdiktion (timezone/countryCode/regionCode)

Normalisierung:

- Trim + Uppercase
- Leere Strings werden als `null` behandelt

Validierung:

- `countryCode` entspricht `^[A-Z]{2}$`
- `regionCode` max. 32 Zeichen (Pattern erweiterbar)

7.9 Docking-Contract (OwnerKey)

Responses enthalten:

- **Company:** `contactOwnerType="COMPANY"`, `contactOwnerId=companyId`
- **Location:** `contactOwnerType="LOCATION"`, `contactOwnerId=locationId`

8. Datenmodell (Persistenz)

8.1 Company

- `companyId`, `name`, `displayName`, `mainLocationId`, `timezone`, `locale`, `logoFileRef`
- **Audit:** `createdAt`, `createdBy`, `modifiedAt`, `modifiedBy`
- `version`

8.2 Location

- `locationId`, `companyId`, `name`, `locationCode`
- `timezone`, `countryCode`, `regionCode`
- `status`, `closedAtUtc`, `closedBy`, `closedReason`
- **Audit + version**

8.3 DeletionTombstone

- `deletionId`, `companyId`, `state` (`IN_PROGRESS` | `FAILED` | `COMPLETED`)
- `requestedAtUtc`, `requestedBySub`
- **optional:** `correlationId`, `idempotencyKey`, `lastError`, `ackServices[]`

8.4 OutboxEvent (optional)

- `eventId`, `eventType`, `occurredAtUtc`
- `aggregateType`, `aggregateId`, `payloadJson`
- `status`, `retryCount`

9. Idempotency-Regeln

Idempotency-Key wird verwendet für:

- `POST /api/v1/companies`
- `PUT /api/v1/companies/{companyId}/headquarter`
- `DELETE /api/v1/companies/{companyId}`

Regel:

- gleicher Key + gleicher Endpoint + gleicher Tenant + gleicher Payload-Hash → gleiche Antwort
- gleicher Key, abweichender Payload-Hash → 409 mit Code `IDEMPOTENCY_CONFLICT`

10. Fehlerformat, Statuscodes, Fehlercodes

10.1 Fehlerformat (API Error)

Fehlerantworten sind JSON und enthalten:

- `timestamp` (ISO-8601 UTC)
- `status` (HTTP Status)
- `code` (String)
- `message` (String)
- `path` (String)
- optional `correlationId`
- optional `details`

10.2 Statuscodes

- 400, 401, 403, 404, 409, 500

10.3 Fehlercode-Mapping (Fehlernummer, Fehlermeldung, Auslöser)

Fehlernummer	code	message (exakt)	Erscheint, wenn ...
400	VALIDATION_ERROR	Request validation failed	Request verletzt Validierung; Details enthalten Feldfehler.
400	INVALID_COUNTRY_CODE	Invalid countryCode (expected ISO-3166-1 alpha-2)	countryCode entspricht nach Normalisierung nicht <code>^[A-Z]{2}\$</code> .
400	INVALID_REGION_CODE	Invalid regionCode	regionCode verletzt Längen-/Zeichenregeln.
401	UNAUTHENTICATED	Authentication required	Authorization Header fehlt oder Tokenvalidierung

Fehlernummer	code	message (exakt)	Erscheint, wenn ...
			scheitert (Signatur/exp/iss).
401	UNAUTHENTICATED	wrong audience	aud enthält nicht company-service.
401	UNAUTHENTICATED	missing claim: tenant_id	tenant_id fehlt (bei tenant-scharfen Endpunkten).
401	UNAUTHENTICATED	missing claim: scp	scp fehlt oder ist leer.
401	UNAUTHENTICATED	invalid subject_type	subject_type ist nicht USER oder SERVICE.
403	FORBIDDEN	Access denied	Token ist gültig, Zugriff wird abgelehnt.
403	SCOPE_MISSING	Required scope is missing	Token enthält nicht den für die Route definierten Scope.
403	TENANT_MISMATCH	tenant mismatch	X-Tenant-Id weicht von tenant_id ab oder Pfad-/DB-Tenantprüfung schlägt fehl (No-Data-Leak ggf. ausgenommen).
404	COMPANY_NOT_FOUND	Company not found	Company existiert nicht oder ist fachlich nicht sichtbar.
404	LOCATION_NOT_FOUND	Location not found	Location existiert nicht oder ist fachlich nicht sichtbar; No-Data-Leak kann Fremdtenant als „nicht gefunden“ behandeln.
404	DELETION_IN_PROGRESS	Company deletion in progress	DeletionTombstone steht auf IN_PROGRESS.
409	IDEMPOTENCY_CONFLICT	Idempotency-Key already used with a different payload	Idempotency-Key wurde bereits mit anderer Payload genutzt.
409	VERSION_CONFLICT	Version conflict	Optimistic Locking schlägt fehl (version veraltet).
409	HEADQUARTER_MUST_BE_OPEN	Headquarter must be OPEN	Headquarter wird auf eine CLOSED Location gesetzt.
409	CANNOT_CLOSE_HEADQUARTER	Headquarter cannot be closed	Close auf die aktuelle Headquarter-Location.
409	LAST_OPEN_LOCATION_REQUIRED	At least one OPEN location is required	Close/Delete würde dazu führen, dass keine OPEN Location verbleibt.
500	INTERNAL_ERROR	Unexpected internal error	Unerwarteter Fehler (DB/Runtime/Serialization).

11. Betrieb / Deployment / Konfiguration

11.1 Serviceeigene Ports (nur Defaults, ausschließlich per ENV konfigurierbar)

Der company-service bindet folgende Ports:

11.1.1 API/Application Port

- Spring Property: `server.port`
- ENV: `SERVER_PORT`
- Default: **8110**

11.1.2 Management/Actuator Port

- Spring Property: `management.server.port`
- ENV: `MANAGEMENT_SERVER_PORT`
- Default: **8181**

11.1.3 Optionale Ports (nur Dev/Test; Default deaktiviert)

Diese Ports werden nur gebunden, wenn entsprechende Features aktiviert werden:

- Debug (JDWP): ENV `DEBUG_PORT` (Default: deaktiviert)
- Remote JMX: ENV `JMX_PORT` und `JMX_RMI_PORT` (Default: deaktiviert)

11.1.4 Regel bei parallelem Betrieb mehrerer Services auf einem Host

Wenn mehrere Services gleichzeitig auf demselben Host laufen, müssen `SERVER_PORT` und `MANAGEMENT_SERVER_PORT` pro Service eindeutig sein.

11.2 Pfad-Konfiguration (nur per ENV)

- API Context Path:
 - Property: `server.servlet.context-path`
 - ENV: `SERVER_SERVLET_CONTEXT_PATH`
 - Default: `/api/v1`
- Actuator Base Path:
 - Property: `management.endpoints.web.base-path`
 - ENV: `MANAGEMENT_ENDPOINTS_WEB_BASE_PATH`
 - Default: `/actuator`

11.3 JWT Resource Server – Konfiguration (nur per ENV)

- JWKS:
 - Property: `spring.security.oauth2.resourceserver.jwt.jwk-set-uri`
 - ENV: `SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_JWK_SET_URI`
- Issuer:
 - Property: `spring.security.oauth2.resourceserver.jwt.issuer-uri`
 - ENV: `SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_ISSUER_URI`
- Erwartete Audience:
 - Property: `security.jwt.audience` (Service-spezifisch)
 - ENV: `SECURITY_JWT_AUDIENCE`
 - Default: `company-service`
- Clock Skew:
 - Property: `security.jwt.clock-skew-seconds`
 - ENV: `SECURITY_JWT_CLOCK_SKEW_SECONDS`
 - Default: 60

11.4 Umgebungsvariablen

Variable	Default	Beispiel	Beschreibung
SPRING_PROFILES_ACTIVE	local	prod	Profilsteuerung (Swagger nur local)
SERVER_PORT	8110	8110	API/Application Port
MANAGEMENT_SERVER_PORT	8181	8181	Management/Actuator Port
SERVER_SERVLET_CONTEXT_PATH	/api/v1	/api/v1	API Context Path
MANAGEMENT_ENDPOINTS_WEB_BASE_PATH	/actuator	/actuator	Actuator Base Path
COMPANY_DB_HOST	localhost	mariadb	DB Host
COMPANY_DB_PORT	3306	3306	DB Port
COMPANY_DB_NAME	company	company	DB Name
COMPANY_DB_USER	root	company_user	DB User
COMPANY_DB_PASSWORD	(leer)	secret	DB Passwort
SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_JWK_SET_URI	(leer)	http://auth-service:8080/.well-known/jwks.json	JWKS URL
SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_ISSUER_URI	(leer)	http://auth-service:8080/api/v1/auth	Issuer URI
SECURITY_JWT_AUDIENCE	company-service	company-service	Erwartete Audience (aud)
SECURITY_JWT_CLOCK_SKEW_SECONDS	60	60	Clock Skew Sekunden
COMPANY_NON_LEAK_404	false	true	No-Data-Leak bei Resource-by-Id
COMPANY_OPS_SCOPE	ops:read	ops:read	Scope für geschützte Actuator Endpunkte
COMPANY_OPS_REQUIRE_SERVICE_SUBJECT	true	true	Geschützte Ops-Endpunkte nur mit Service-Token
COMPANY_SWAGGER_ENABLED	true (nur local)	false	Swagger/OpenAPI aktiviert/deaktiviert
DEBUG_PORT	(deaktiviert)	5005	JDWP Debug Port (nur Dev/Test)
JMX_PORT	(deaktiviert)	9010	Remote JMX Port (nur Dev/Test)
JMX_RMI_PORT	(deaktiviert)	9011	Remote JMX RMI Port (nur Dev/Test)

12. Tests und Abnahmekriterien

12.1 Tests

- Integrationstests REST + DB (Testcontainers MariaDB)

- Security-Tests:
 - JWT-Validierung (`aud`, `iss`, `exp`, `JWKS`)
 - Scope-Prüfung pro Endpoint
 - Tenant-Isolation (`companyId==tenant_id`, Location-by-Id über `location.companyId`)
 - X-Tenant-Id Konsistenzcheck
 - No-Data-Leak Verhalten (404 bei Fremd-ID, wenn aktiviert)
- Headquarter-Tests:
 - Setzen/Abfragen, Konsistenz über `mainLocationId`
 - Setzen auf `CLOSED` → 409
 - Close auf Headquarter → 409
- Location-Regeln:
 - Close der letzten `OPEN` Location → 409
 - Reopen setzt Status zurück auf `OPEN`
- Idempotency-Tests:
 - gleicher Key → gleiche Antwort
 - anderer Payload → 409
- Deletion Workflow Tests:
 - `DELETE` → 202 + Status
 - `GET company` während `IN_PROGRESS` → 404

12.2 Abnahmekriterien

- Build erfolgreich (`mvn clean package`)
- Fachliche Endpunkte sind IAM-geschützt (JWT + Scope + Tenant-Prüfungen; Default-Deny aktiv)
- Serviceeigene Ports sind ausschließlich per ENV konfigurierbar (Defaults dokumentiert)
- `SERVER_PORT` und `MANAGEMENT_SERVER_PORT` sind bei parallelem Host-Betrieb je Service eindeutig
- Actuator ist erreichbar unter `http://<host>:${MANAGEMENT_SERVER_PORT}/actuator/...`
- Swagger/OpenAPI: local aktiv, nicht-local deaktiviert
- Headquarter-API funktioniert (GET/PUT, Validierungen)
- Jurisdiktion-Felder werden normalisiert/validiert
- Idempotency für definierte Endpunkte aktiv
- Löschworkflow liefert 202 und Statusabfrage liefert Workflow-State

(separat, empfohlen)

Effektive Basis-URL: `http://<host>:8080/api/v1`